

OMI Lazy Distributed Evaluation Proof Book

Purpose

This document defines how OMI supports lazy distributed evaluation without making the port layer execute, connect, transmit, validate, receipt, or accept state.

It is the readable companion for aligning:

```
OMI workbook notation
omi-port route candidates
local evaluator bindings
Coq proof authority
OMI-ISA executable witnesses
receipt records
```

The central claim is narrow:

OMI can name a distributed evaluation candidate before that evaluation occurs.
The candidate is expressed as a bounded source-target route, optionally bound to a local or remote evaluator, and preserved as dormant until downstream validation and receipt authority acts.

This is not a claim that `omi-port` performs distributed computation.

This is not a claim that a route is a connection.

This is not a claim that a binding is authority.

The purpose of `omi-port` is to declare where evaluation could happen, not to perform it.

Final Doctrine

```
OMI-Port names lazy evaluation.
It does not perform evaluation.

A PortRoute names a possible source-target relation.
A PortBinding names a possible evaluator for that route.
A PortPipe would activate the relation, and is outside v0.

Coq proves.
Canon names.
```

```
OMI workbook notation annotates.  
OMI-Port derives dormant candidates.  
OMI-Lisp may declare.  
OMI-ISA may witness.  
Validation decides.  
Receipt records.
```

1. Claim Status

Every statement in this document belongs to one of four classes.

Status	Meaning
Proved	An active Coq theorem compiles without forbidden assumptions and passes the strict proof gate.
Implemented	Runtime code or tests demonstrate behavior, but no refinement proof yet connects it fully to the Coq model.
Defined model	The object, rule, or boundary is precise enough to implement and test.
Interpretation	A name, metaphor, diagram, or explanation used to make the model readable.

This prevents four mistakes:

```
a route is not a connection  
a binding is not execution  
a deterministic candidate is not validation  
a receipt is not a proof
```

2. Authority Boundary

OMI separates one relation pipeline into distinct responsibilities:

```
OMI workbook notation  
  annotates a page, block, entity, or local object  
  
OMI-Port  
  derives a dormant source-target route candidate
```

PortBinding
associates that route with a possible evaluator

Omicron
frames and resolves the candidate when invoked downstream

Tetragrammatron
validates lawful closure

Metatron
interpolates or projects accepted decisions without changing them

OMI-ISA
executes bounded witness fixtures

Receipt
records accepted replay or decision

The authority rule is:

Declaration is not authority.
Connection requires authority.
Validation decides.
Receipt records.

`omi-port` is therefore not a runtime executor.

It is a dormant declaration layer.

3. Why Lazy Evaluation Needs OMI-Port

A distributed system often wants to say:

this source may be evaluated by that target

without saying:

this source has already been evaluated

or:

```
this source is connected to that target
```

or:

```
this target has authority over the source
```

That is the exact role of `omi-port`.

It gives OMI a safe pre-evaluation form:

```
source scope
target scope
gauge
route candidate
optional evaluator binding
all authority flags false
```

The canonical statement is:

```
OMI-Port declares an evaluable relation.
It does not evaluate the relation.
```

4. Objects

4.1 PortScope

A `PortScope` names a bounded source or target.

Default printable grammar:

```
<uri>#/<prefix>
```

Examples:

```
file:///repo/docs/spec.md#/64
obsidian://vault/Page#block-id#/48
lora://915/node/alpha#/16
serial://ttyUSB0#/8
```

```
dom://document/main/button#/32
canvas://board/card/7#/24
local://evaluator/null-ring#/32
proof://omi-axioms/NullRingCloses#/64
isa://omi-isa/programs/null_ring.omi#/64
```

The URI names.

The prefix scopes.

Neither one grants authority.

Canonical rule:

```
URI names.
CIDR scopes.
OMI routes.
Validation accepts or rejects.
```

4.2 PortTensor

A `PortTensor` derives a route from source and target scopes under an F^* gauge:

```
PortTensor_G(source, target) -> PortRoute
```

The gauge selects a deterministic transform dialect.

It does not select truth.

Examples:

```
F0 contradiction candidate
F6 XOR-like difference candidate
F8 AND-like intersection candidate
F9 XNOR-like equivalence candidate
FB source-to-target implication candidate
FD target-to-source converse candidate
FE OR-like union candidate
FF canonical closure candidate
```

Canonical rule:

```
F* selects transform dialect.  
It does not validate the route.
```

4.3 PortRoute

A `PortRoute` is the derived dormant route candidate.

Folded form:

```
omi---imo
```

Unfolded form:

```
o---o/---/?---?@---@
```

Machine-expanded route topology:

```
o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR
```

The route contains place-value structure.

It does not contain authority.

Required default flags:

```
connects  = 0  
transmits = 0  
mounts    = 0  
accepted  = 0  
validated = 0  
receipted = 0
```

4.4 PortBinding

A `PortBinding` associates a dormant route with a possible evaluator.

It answers:

If this route is later authorized,
which local or distributed evaluator could receive it?

Examples of evaluator scopes:

```
local://function/null-ring-check#/32
proof://omi-axioms/NullRingCloses#/64
isa://omi-isa/test/test_null_ring#/64
worker://browser/semantic-resolver#/40
device://esp32/lora-node-alpha#/16
obsidian://vault/plugin/omi-vault#/32
```

A binding may point to:

- a function name
- a proof name
- a command name
- a note id
- a device id
- a worker id
- a plugin id
- a human-readable workbook entity

But it still does not execute.

Canonical rule:

A PortBinding may point to an evaluator.
Only downstream authority may wake it.

4.5 PortPipe

A `PortPipe` would be an active stream.

It implies one or more of:

- connect
- send
- receive
- mount
- open

```
execute
transmit
```

Therefore:

```
PortPipe is outside omi-port v0.
```

Canonical rule:

```
A binding may be declared.
A pipe must be authorized.
```

5. Workbook Notation Layer

OMI workbook notation is the human-facing annotation layer.

It belongs naturally in Markdown, Obsidian, Org-like outlines, local notes, source comments, and documentation blocks.

It does not carry arbitrary functions.

It carries:

```
place-value address
gauge frame
relation witness
nearest attestation rule
optional local binding reference
```

A page can say:

```
## Null Ring Fixture
```

```
This block describes the executable NULL Ring witness.
```

```
%%omi
address: omi---imo/null-ring
surface: o----o/---/?---?@---@
gauge: FF
frame: FS.GS.RS.US
```

```
attestation: nearest-heading
source: obsidian://vault/Null Ring Fixture#/64
target: isa://omi-isa/programs/null_ring.omi#/64
binding: local://test/test_null_ring#/64
state: dormant
%%
```

This annotation does not execute the test.

It declares a route candidate and an optional evaluator binding.

6. Nearest Attestation Rule

OMI workbook annotations are interpreted by nearest attestation.

An annotation may be scoped to:

```
nearest heading
nearest block id
nearest paragraph
nearest fenced code block
nearest file
nearest entity tag
nearest explicit anchor
```

The rule is:

```
If an annotation does not explicitly name its subject,
the nearest attested structure is the subject.
```

Example:

```
### Tetragrammatron Governor

The Tetragrammatron separates three clocks, four offsets, and five governors.

%%omi
address: omi---imo/tetragrammatron/polyharmonic-governor
attestation: nearest-heading
source: obsidian://vault/Tetragrammatron Governor#/64
target: proof://omi-axioms/MiquelIncidenceBalancesFlags#/64
```

```
binding: isa://omi-isa/test/test_tetragrammatron_witness#/64
state: dormant
%%
```

The annotation binds to the heading because `attestation: nearest-heading`.

7. Nested Coordinate Notation

The workbook notation may also be expressed structurally:

```
omi-o
  in frame FS
  /
    in frame GS
    ?
      in frame RS
      @
        at frame US
        @
      ?
    /
  .o-imo
```

This is not a requirement that the marks visually touch.

It is nested coordinate structure.

Interpretation:

o	source/object frame
/	plane or group separator
?	witness/query/receipt-facing frame
@	address/unit/binding frame

Gauge coordinates:

```
FS
GS
RS
US
```

Delineation hierarchy:

```
o---o
/---/
?---?
@---@
```

Canonical rule:

```
Coordinates need not touch.
They must nest.
Nearest attestation resolves them.
```

8. Lazy Distributed Evaluation Pipeline

The complete dormant pipeline is:

```
workbook page, block, note, object, proof, or device reference
↓
nearest OMI annotation
↓
source PortScope
↓
target PortScope
↓
F* gauge selection
↓
PortTensor_G(source, target)
↓
PortRoute candidate
↓
optional PortBinding to evaluator scope
↓
dormant handoff
↓
future validation authority
↓
future receipt authority
↓
optional authorized execution outside omi-port v0
```

Nothing in this pipeline connects.

Nothing in this pipeline transmits.

Nothing in this pipeline validates.

Nothing in this pipeline receipts.

It only names and preserves a candidate.

9. C Shape

A minimal C shape for `PortBinding` may be:

```
typedef struct {
    char uri[256];
    unsigned prefix_len;
} OMI_PortScope;

typedef struct {
    unsigned char gauge;

    OMI_PortScope source;
    OMI_PortScope target;

    unsigned short fs;
    unsigned short gs;
    unsigned short rs;
    unsigned short us;

    unsigned short fs_i;
    unsigned short gs_i;
    unsigned short rs_i;
    unsigned short us_i;

    unsigned int reg;
    unsigned int stack;
    unsigned int car;
    unsigned int cdr;

    int connects;
    int transmits;
    int mounts;
    int accepted;
```

```

    int validated;
    int receipted;
} OMI_PortRoute;

typedef struct {
    OMI_PortRoute route;
    OMI_PortScope evaluator;

    int dormant;

    int connects;
    int transmits;
    int mounts;
    int accepted;
    int validated;
    int receipted;
} OMI_PortBinding;

```

Required creation behavior:

```

dormant    = 1
connects   = 0
transmits  = 0
mounts     = 0
accepted   = 0
validated  = 0
receipted  = 0

```

Required non-behavior:

```

do not open
do not connect
do not send
do not receive
do not mount
do not execute
do not validate
do not receipt

```

10. Proof Boundary

The proof rail belongs to `omi-axioms`.

The port layer is not the proof authority.

Correct propagation:

```
Coq theorem
  proves a bounded invariant

OMI canon
  names the invariant

OMI workbook notation
  annotates the place where the invariant is discussed

OMI-Port
  derives dormant source-target route candidates

PortBinding
  points to a possible evaluator

OMI-ISA
  executes bounded witness fixtures

runtime test
  checks implementation behavior against expected examples

receipt
  records accepted replay or decision
```

No downstream layer retroactively proves the theorem.

Conversely, Coq does not prove C, JavaScript, firmware, LoRa, filesystem, or browser behavior unless a refinement proof connects that implementation to the formal model.

11. Current Proof Anchors

The active proof book provides anchors for these families:

```
Null Ring closure
  byte-ring XOR witnesses
  finite closed path closure
  256-bit self-cancellation

Earned control bands
```

invisible bytes before readable language
Omicron resolver envelope
FS/GS/RS/US control boundaries

Polybius diagonal closure
D+ and D- XOR closure
0x1E, 0x3C, 0x78 sums
240-state active byte surface

Tetragrammatron authority
lawful/unlawful validation decision
finite witness sequence closure
malformed frame rejection

Metatron interpolation
preserves validation decision
cannot make unlawful lawful

Finite truth-table scale
 2^n input configurations
 $2^{(2^n)}$ Boolean laws
fixed-width lookup model

Incidence geometry
Fano finite incidence
Miquel flag balance
3-clock, 4-offset, 5-governor frame
 $5 + 6 = 11$ oversight arithmetic

These proofs support the lazy evaluation architecture by fixing the boundaries:

framing is not validation
validation is not projection
projection is not receipt
binding is not execution

12. Implementation Tests

omi-port tests should verify:

PortScope parsing
valid URI-CIDR parses
malformed URI-CIDR fails

prefix is scope width, not gauge

PortTensor transform

same source + target + gauge gives same route
different source changes route
different gauge changes route
all authority flags remain zero

PortRoute notation

folded omi---imo recognized
unfolded o---o/---/?---?@---@ recognized
FS/GS/RS/US topology preserved
LL/MM/NN absent from public API

PortBinding

route + evaluator scope creates dormant binding
dormant == 1
connects == 0
transmits == 0
mounts == 0
accepted == 0
validated == 0
receipted == 0
creation is deterministic
binding does not call network, filesystem, or runtime APIs

Forbidden library API scan:

```
grep -rn 'connect\|send\|recv\|socket\|open\|mount' src/ include/ || true
```

Expected result:

```
no matches in library code
```

Authority flag scan:

```
grep -rn 'authority\.(accepted\|validated\|receipted\|connects\|transmits\|mounts)\s*=\s*[1-9]' src/ || true
```

Expected result:

no matches

13. Release Boundary

Before lazy distributed evaluation work is promoted, the following must remain true:

```
make clean && make test passes
make hs-check passes or cleanly skips
no forbidden active I/O APIs in library code
no sibling repo imports
no authority flags set by omi-port
golden route outputs do not drift without version bump
PortBinding remains dormant
PortPipe remains outside v0
```

Golden outputs freeze deterministic candidate behavior only.

They do not freeze truth.

They do not validate.

They do not receipt.

They do not create identity.

14. Example A: Workbook Block to Dormant Route

Workbook note:

```
## Null Ring

The byte-ring closes by XOR reduction.

%%omi
address: omi---imo/null-ring
source: obsidian://vault/Null Ring#heading#/64
target: proof://omi-axioms/NullRingCloses#/64
gauge: FF
attestation: nearest-heading
```

```
state: dormant
%%
```

Port interpretation:

```
source = obsidian://vault/Null Ring#heading#/64
target = proof://omi-axioms/NullRingCloses#/64
gauge  = FF
route  = PortTensor_FF(source, target)
flags  = all zero
```

No proof has been run.

No function has been executed.

No receipt has been accepted.

15. Example B: Binding to an ISA Fixture

Annotation:

```
%%omi
address: omi---imo/null-ring
source: proof://omi-axioms/NullRingCloses#/64
target: isa://omi-isa/programs/null_ring.omi#/64
binding: local://omi-isa/test_null_ring#/32
state: dormant
%%
```

Meaning:

```
The proof anchor may be associated with an executable ISA witness.
The binding names the witness.
The binding does not run the witness.
```

Only a downstream authorized runner may execute it.

16. Example C: Distributed Device Candidate

Annotation:

```
%%omi
address: omi---imo/sensor/attestation
source: obsidian://vault/Sensor Note#block-7#/48
target: lora://915/node/alpha#/16
binding: device://esp32/lora-alpha/evaluator#/16
gauge: FB
state: dormant
%%
```

Meaning:

A source note may be routed toward a LoRa device evaluator.
The gauge reads as source-to-target implication-like.
The route is still dormant.
No radio transmission occurs.

17. What This Does Not Claim

This model does not claim:

```
OMI-Port performs evaluation
OMI-Port connects distributed machines
OMI-Port sends messages
OMI-Port opens files
OMI-Port mounts devices
OMI-Port validates frames
OMI-Port receipts state
OMI-Port proves Coq theorems
OMI-Port replaces local runtimes
```

It claims only:

OMI-Port can name lazy distributed evaluation candidates
without activating them.

18. Glossary

Term	Meaning
Binding	Dormant association between a route and a possible evaluator.
Candidate	A named possibility that has not been accepted.
Dormant	Declared but inactive; no connection, transmission, validation, or receipt.
Evaluator	A local or remote object that may later interpret a binding if authorized.
Gauge	F* transform dialect selector.
PortPipe	Active stream; outside <code>omi-port</code> v0.
PortRoute	Derived OMI-IMO route candidate.
PortScope	URI-CIDR source, target, or evaluator scope.
PortTensor	Deterministic transform from source and target to route candidate.
Receipt	Accepted record of validation or replay.
Workbook notation	Markdown/Obsidian annotation layer for OMI place-value addresses.

19. Final Canon

The workbook annotates place.
OMI-Port derives dormant route candidates.
PortBinding names possible evaluators.
No binding is execution.
No route is connection.
No gauge is truth.
No projection is receipt.

Coq proves the invariant.
Canon names the invariant.
OMI workbook notation cites the invariant.
OMI-Port routes the candidate.
OMI-ISA may witness the bounded execution.
Validation decides.
Receipt records.

Shortest form:

OMI-Port is where lazy distributed evaluation is named,
not where it is performed.